

```

1  (*
2                                     CS51 Lab 7
3                               Modules and Abstract Data Types
4  *)
5
6  (*
7                                     SOLUTION
8  *)
9
10 (* Objective: This lab practices concepts of modules, including files
11 as modules, signatures, and polymorphic abstract data types.
12
13 There are three total parts to this lab. Please refer to the following
14 files to complete all exercises:
15
16     lab7_part1.ml -- Part 1: Implementing modules
17 -> lab7_part2.ml -- Part 2: Files as modules (this file)
18     lab7_part3.ml -- Part 3: Polymorphic abstract types and
19                           Interfaces as abstraction barriers
20 *)
21
22 (*=====
23 Part 2: Files as modules
24
25 A useful feature of OCaml is that it *automatically* wraps all of the
26 functions and values that are defined in a single file into a module
27 named after that file. The module name is the name of the file with
28 the first letter capitalized. This functionality is in addition to the
29 manual definition of modules as you've just used in Part 1, and it is
30 a convenient way of separating code into separate namespaces when
31 writing a large program.
32
33 There are other source files included in this lab, other than the
34 `lab7_partn.ml` files. The file `color.ml` contains an implementation
35 of a system for managing colors. (Recall from lab 5 the idea of colors
36 as consisting of values for three color channels -- red, green, and
37 blue.) Take a look at `color.ml` to see what functions and values it
38 contains, including a type for colors, and a means for converting
39 values for the three color channels into the abstract `color` type,
40 extracting the channels individually from colors, and converting some
41 standard color names to this color representation.
42
43 With the exception of Exercise 2A, you will need to modify *only*
44 the file `color.ml` to complete the exercises below.
45
46     A digression on accessing other modules:
47
48     You'll want to test this part of the lab using `ocamlbuild`, for
49     instance, with
50
51         % ocamlbuild -use-ocamlfind lab7_part2.byte
52         % ./lab7_part2.byte

```

The ``ocamlbuild`` command should automatically find modules that you've written that reside in the same directory as your source, compile those additional files, and link them to your compiled program. You can then access functions from those files under the module name, which (again) is the name of the file with the first letter capitalized. For instance, if ``color.ml`` is in the same directory as ``lab7_part2.ml`` (which it probably is), ``ocamlbuild`` will find it and use it, since it is referenced through expressions like ``Color.red`` and the like.

On the other hand, if you're testing with a top-level REPL, like `utop` or `ocaml`, it will **not** automatically find those modules and evaluate them for you. However, you can inform the REPL about those modules manually yourself with the ``#mod_use`` directive, like this:

```
# #mod_use "color.ml" ;;
```

allowing you to then refer to elements of the module as, for instance,

```
# Color.color_named ;;
- : Color.color_name -> int = <fun>
```

(Note the capitalized module name, as discussed above.)

Keep in mind however that the ``#mod_use`` and ``#use`` directives look only at their argument files, not at any corresponding ``.mli`` files, so the modules being used will **not** be restricted to the signatures in the corresponding ``.mli`` files. For that, you'll have to use the compiler (with ``ocamlbuild`` for instance).

.....
Exercise 2A: Replace the ``0`` in the expression below with an expression that extracts the red channel of the color named ``Red``, naming the result ``red_channel_value``. (The expression will be constructed using values in the ``Color`` module.)
.....*)

```
let red_channel_value : int =
  let open Color in
    red_channel (color_named Red) ;;
```

(* Without the local open, we get the slightly more verbose

```
    let red_channel : int =
      Color.red_channel (Color.color_named Color.Red) ;;
  *)
```

(* Let's investigate one way that a signature can be useful. Although ``color.ml`` contains an implementation of a basic color module, the implementation is unintuitive and obscure -- truly **horrid** in fact. (We did that on purpose.) You will want to change the implementation of

```

107 `color.ml`, rewriting it wholesale. At the same time, you'll want to
108 guarantee to users of the module (like this file itself!) that the
109 functionality from the "outside" point of view stays the same; the way
110 to do this is through module *signatures*.
111
112 .....
113 Exercise 2B: Add a file `color.mli`, in which you define an appropriate
114 signature for the `Color` module. Consider which types and values you
115 want revealed to the user and which you would prefer to be hidden.
116
117     HINT: You'll want users of the module to know that there is a
118     `color` type, but not how it's implemented. You'll want users of
119     the module to know that there is a `color_name` type, and also
120     what its enumerated constructors are, so that users can invoke
121     those colors.
122
123 Once you have `color.mli` implemented, you should still be able to
124 compile `color.ml` and run `color.byte`.
125 .....*)
126
127 (* SOLUTION: The module type for the `Color` module captures the
128 interface we want our color modules to obey. Your `color.mli` file
129 should have the following types and values declared. If you have
130 trouble getting this to work, you can find our documented solution in
131 `color.mli`.
132
133     type color ;;
134     type color_name =
135         | Red
136         | Green
137         | Blue
138         | Orange
139         | Yellow
140         | Indigo
141         | Violet ;;
142     val to_color : int -> int -> int -> color ;;
143     val red_channel : color -> int ;;
144     val green_channel : color -> int ;;
145     val blue_channel : color -> int ;;
146     val color_named : color_name -> color ;;
147 *)
148
149 (*.....
150 Exercise 2C:
151
152 In the file `color.ml`, modify the implementation of a color module as
153 you see fit. Make the design choices that you think would be best for a
154 color module implementation. In particular, you should be able to come
155 up with a solution that is *much* SIMPLER, clearer, and more transparent
156 than the one currently in `color.ml`. (Frankly, I recommend throwing out
157 the `color.ml` contents entirely and starting over.)
158
159 To pass the unit tests, you'll want the RGB values for the colors in the
160 `color_name` type to have the following values from lab 5:

```

```

161
162      R | G | B | Color
163      ---|---|---|-----
164      255 | 0 | 0 | Red
165      0 | 255 | 0 | Green
166      0 | 0 | 255 | Blue
167      255 | 165 | 0 | Orange
168      255 | 255 | 0 | Yellow
169      75 | 0 | 130 | Indigo
170      240 | 130 | 240 | Violet
171
172      .....*)
173
174      (* SOLUTION: See the solution in `color.ml` *)
175
176      (* Here's the payoff: A user who uses the `color` module, by virtue of
177      having to stay within the `color.mli` interface, will not notice *any
178      difference at all* between the two implementations of `color.ml`, the
179      horrid one we provided and the elegant one you've developed. The
180      underlying implementation can be changed any time in any way, so long
181      as the functionality provided stays consistent with the signature.
182
183      Correspondingly, consider the `List` module that you are familiar with
184      by now. You never needed to worry about how the `List` module was
185      implemented in order to use it; you only needed to understand the
186      interface.
187
188      Compartmentalization ftw! *)

```